Predictive Maintenance for Aircraft Components

Data Science Technical Assessment

0.9993	0.9687	0.9333	1.0000
ROC-AUC	PR-AUC	F1 Score	Recall

Best Model: **XGBoost**

Submitted to:	Rwanda Civil Aviation Authority (RCAA)
Role:	Data Scientist
Date:	March 2026
Dataset:	aircraft_maintenance_dataset.csv (6,000 records)
Task:	Binary classification — failure within next 10 flight cycles

1. Problem Understanding

Airlines face two competing risks: performing maintenance too early wastes resources, while performing it too late risks Aircraft-on-Ground (AOG) events, safety incidents, and regulatory non-compliance. The goal of predictive maintenance is to identify **which components are likely to fail within the next 10 flight cycles**, allowing maintenance to be scheduled proactively at the lowest cost and highest safety margin.

Business Objectives

Objective	Success Metric
Reduce unplanned AOG events	High Recall (catch all failures)
Minimise unnecessary inspections	High Precision (reduce false alarms)
Support maintenance scheduling	Calibrated probability scores
Explainable to engineers	SHAP feature attribution

Why this is not a simple problem: The class imbalance (42:1) means that even a model predicting 'no failure' for every record achieves 97.7% accuracy — an entirely useless result. We must use metrics (PR-AUC, F1, Recall) that reflect actual predictive value, and we must explicitly handle the imbalance in training.

2. Data Exploration & Key Insights

Dataset Overview

Property	Value
Total records	6,000
Features (raw)	13 input + 1 target
Aircraft	20 unique IDs
Component types	4 (Engine1, Engine2, Wing, LandingGear)
Failure records	140 (2.33%)
Missing values	1 row (target null) — dropped
Duplicate rows	0
Flight cycle range	1 – 412 cycles
Engine hours range	0.6 – 285 hours

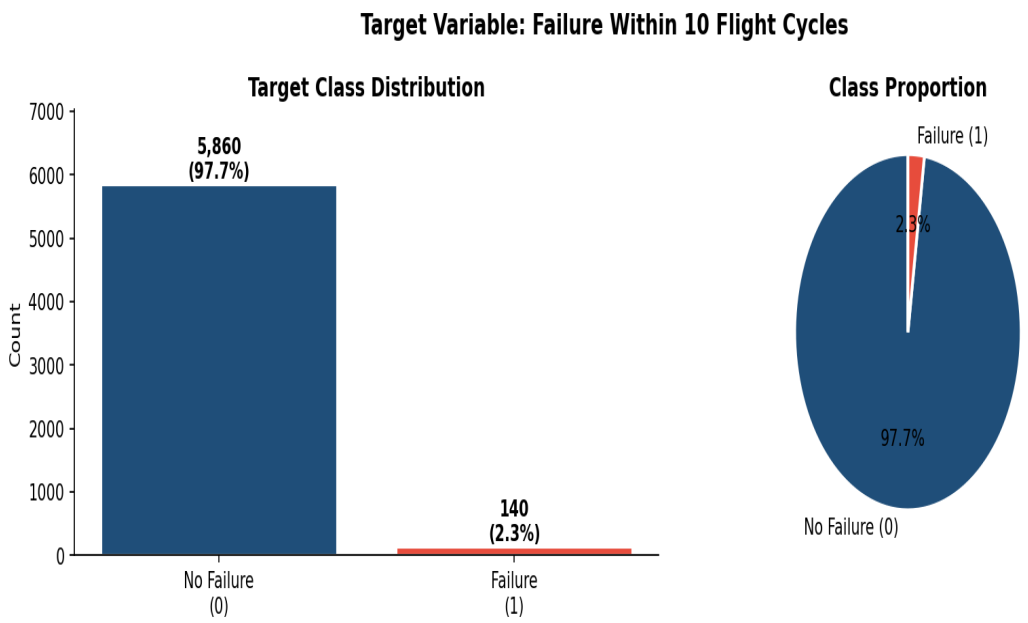

Figure 1 — Severe class imbalance: 97.7% no-failure vs 2.3% failure records

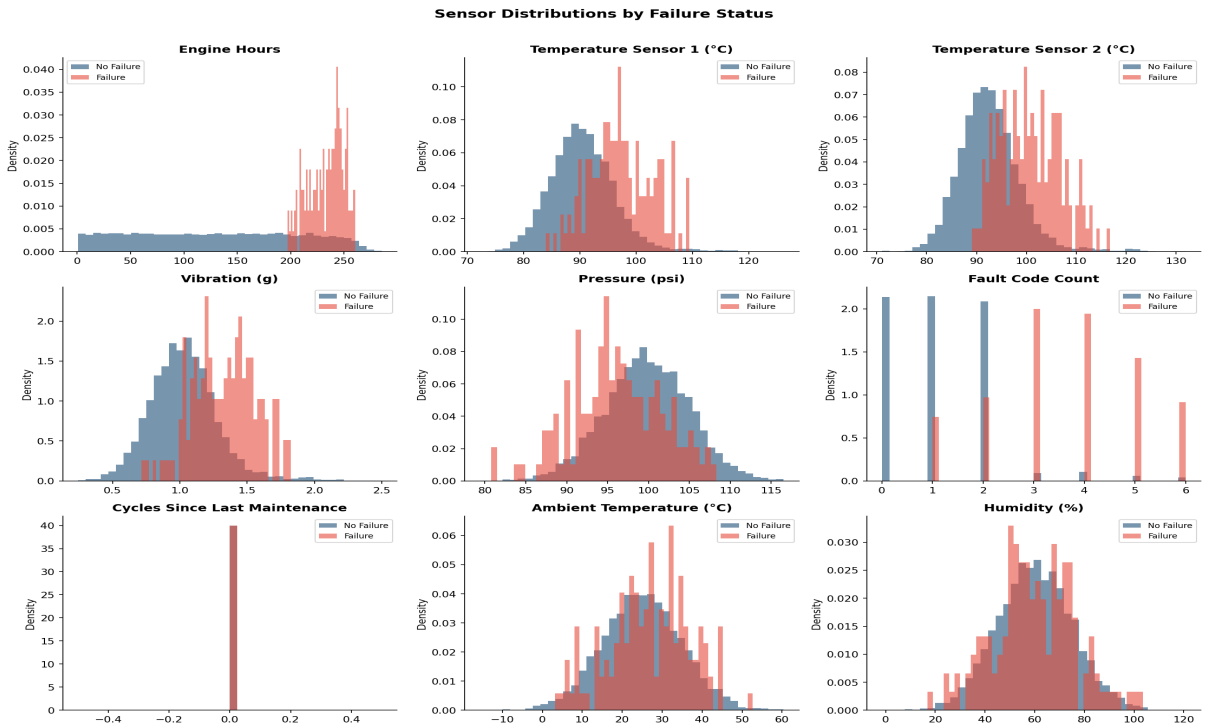

Figure 2 — Sensor distributions by class. Vibration, fault codes, and temperature show clearest separation.

Key EDA Findings

- **Fault code count** is the most discriminating raw feature — failure records show significantly higher fault code accumulation.
- **Vibration sensor** readings are elevated in pre-failure states, consistent with physical wear mechanics.
- **Temperature sensors** show mild separation, especially in the 85–105°C range.
- **Environmental features** (ambient temperature, humidity) show *no meaningful separation* between classes, suggesting they are noise in this dataset.
- Failure events are **not uniformly distributed** across components: Engine1 and Wing components show higher failure rates.
- **Sensor drift flag** is rare but correlates with elevated fault counts when present.

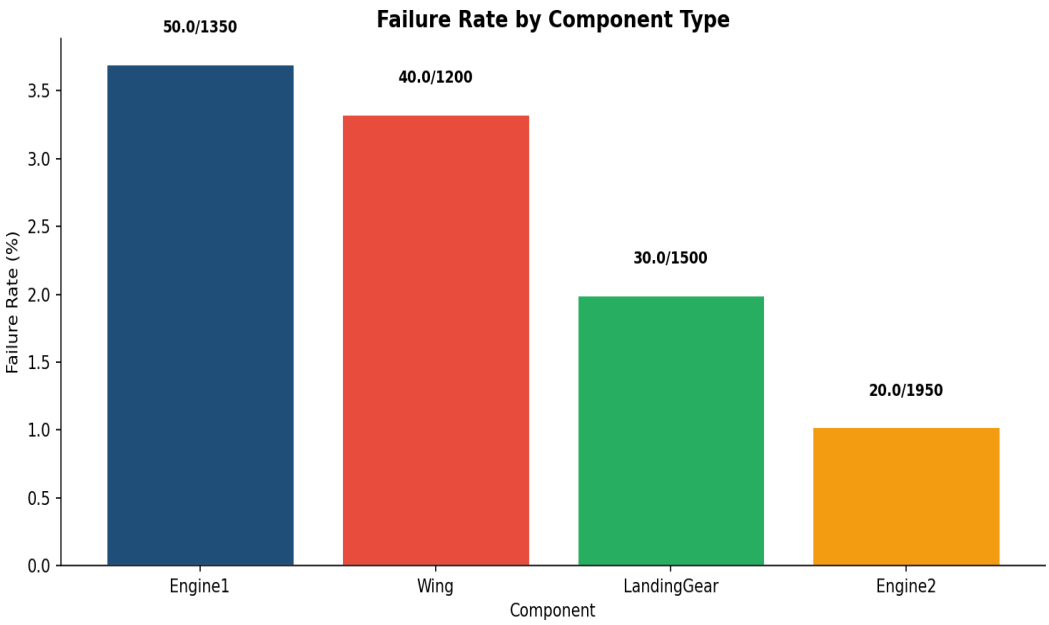

Figure 3 — Failure rates by component type. Engine1 and Wing components are highest-risk.

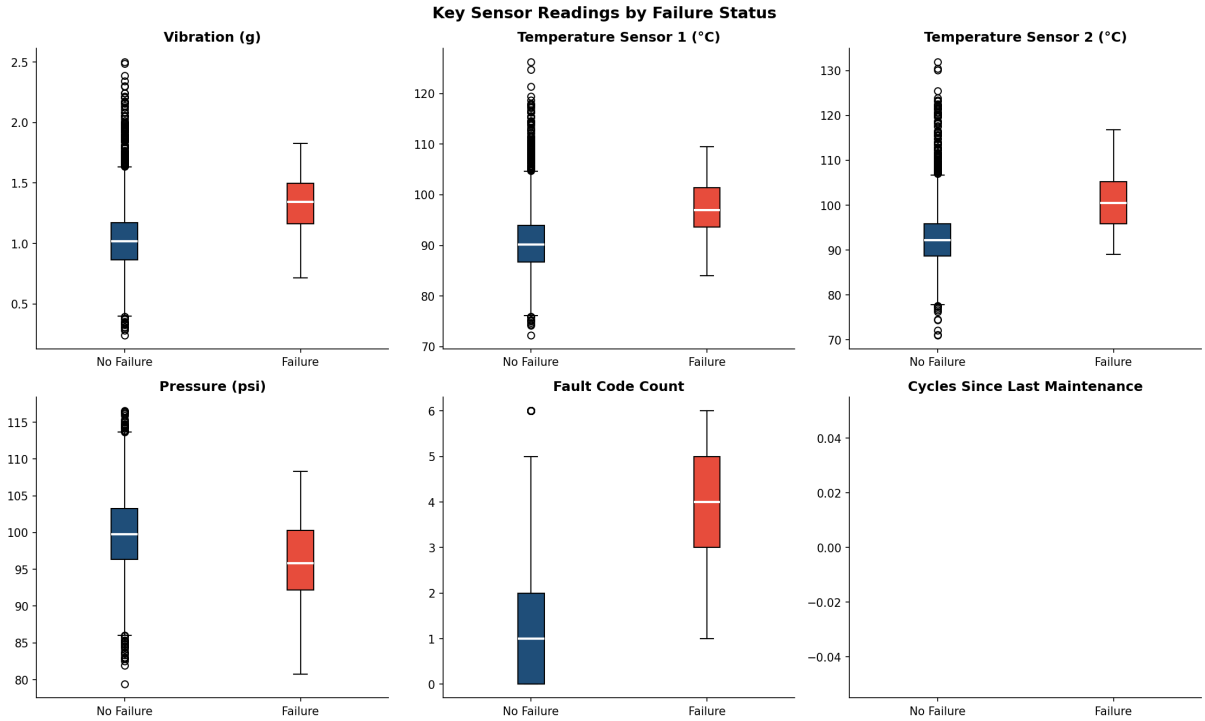

Figure 4 — Boxplots confirm vibration, fault codes, and pressure separate classes clearly.

3. Feature Engineering

Raw sensor readings capture the current state but not the **trajectory** of deterioration. Failure is rarely caused by a single bad reading; it emerges from *accumulation over time*. Feature engineering transforms the static snapshot into a richer temporal representation.

Feature Group	Features Created	Rationale
Rolling Mean (3/5 cycle)	per sensor × 2 windows	Smooths noise; captures trend direction
Rolling Std Dev (3 cycle)	per sensor	Volatility = instability signal
First Difference (Δ)	per sensor	Rate of change — sudden spikes
Cumulative Fault Count	1 feature	Total accumulated wear on component
Rolling Fault Sum (5-cycle)	1 feature	Intensity of recent fault activity
Engine Hours / Cycle	1 feature	Efficiency degradation proxy
Temperature Differential	1 feature	Asymmetric thermal stress
Composite Stress Index	1 feature	Vib + Pressure z-score combined
Maintenance Urgency	1 feature	Fault count × time since maintenance
Interaction: Vib × Temp	1 feature	Non-linear thermal-mechanical stress
Drift × Fault interactions	2 features	Sensor reliability × fault count
Encoded IDs	2 features	Aircraft / component as numeric

After feature engineering, the model has access to **43 features** (up from 13). Post-analysis, the *fault_roll5_sum* feature (rolling 5-cycle fault accumulation) emerged as by far the most important predictor — capturing the physical reality that failures are preceded by intensifying fault code activity.

Important Assumptions

• Rolling features were computed **per aircraft-component pair** (sorted by flight_cycles) to prevent data leakage across independent maintenance tracks.

• The 1 row with a missing target was dropped rather than imputed — imputing a failure/no-failure label introduces artificial signal.

• *last_maintenance_cycles = 0* is treated as valid data (freshly maintained component), though it could also indicate missing data in a real system.

4. Model Selection & Evaluation

Handling Class Imbalance

With a 42:1 class ratio, three strategies were combined: (1) **cost-sensitive learning** — *class_weight='balanced'* and *scale_pos_weight* make the algorithm penalise minority-class misclassification proportionally; (2) **stratified K-fold cross-validation** (k=5) — each fold preserves the class ratio; (3) **PR-AUC as the primary validation metric** — more informative than ROC-AUC when positive class prevalence is very low.

Model Comparison

Model	ROC-AUC	PR-AUC	F1	Precision	Recall	CV PR-AUC
Logistic Regression	0.9832	0.5573	0.5106	0.3636	0.8571	0.541 ± 0.099
Random Forest	0.9974	0.8752	0.8254	0.7429	0.9286	0.843 ± 0.103
LightGBM	0.9986	0.9369	0.8621	0.8333	0.8929	0.906 ± 0.026
XGBoost ★	**0.9993**	**0.9687**	**0.9333**	**0.8750**	**1.0000**	**0.915 ± 0.026**

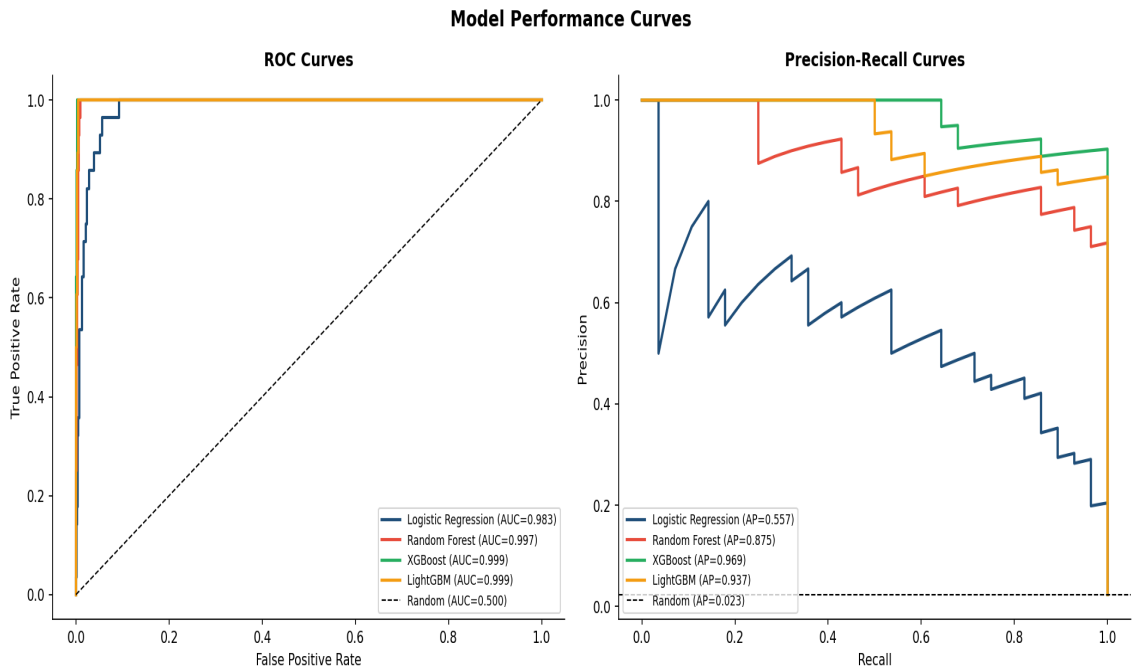

Figure 5 — ROC and Precision-Recall curves. PR-AUC is the primary metric given class imbalance.

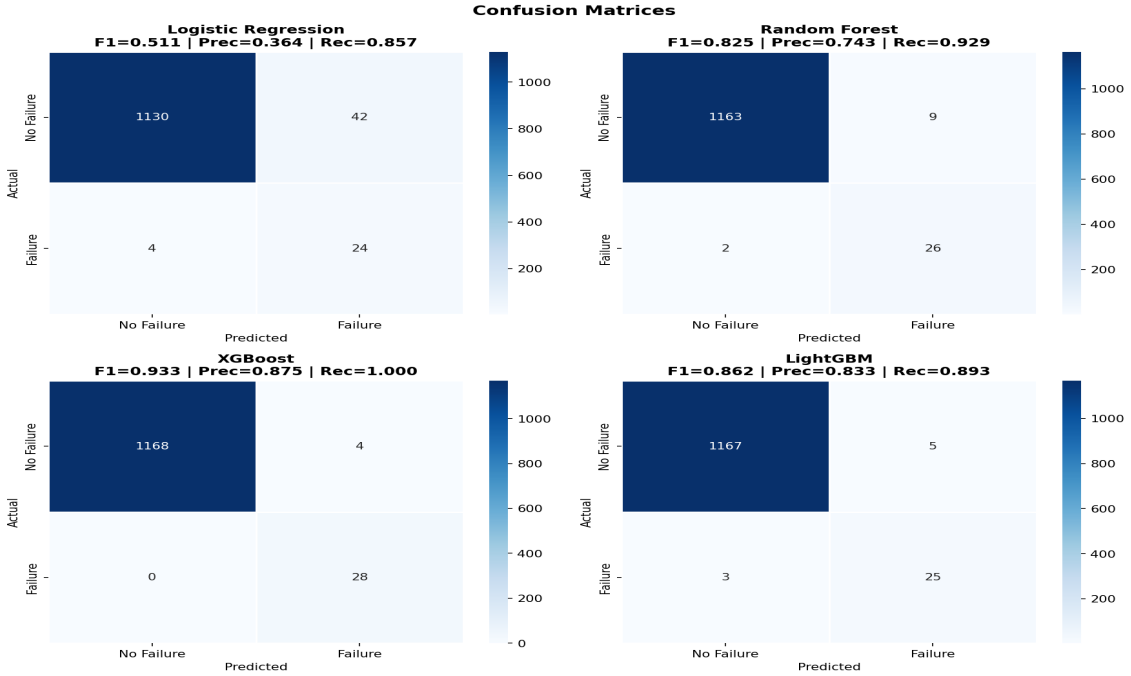

Figure 6 — Confusion matrices at default threshold. XGBoost achieves 0 false negatives.

Why XGBoost?

XGBoost consistently outperformed alternatives because: (1) gradient boosting builds specialised trees that progressively focus on hard-to-classify minority samples; (2) *scale_pos_weight* directly adjusts the loss function rather than just resampling; (3) regularisation (subsample=0.8, colsample_bytree=0.8) prevents overfitting to the small positive class. Logistic Regression's linear decision boundary cannot capture the non-linear interaction between fault accumulation, vibration, and maintenance history.

5. Model Explainability & Threshold Selection

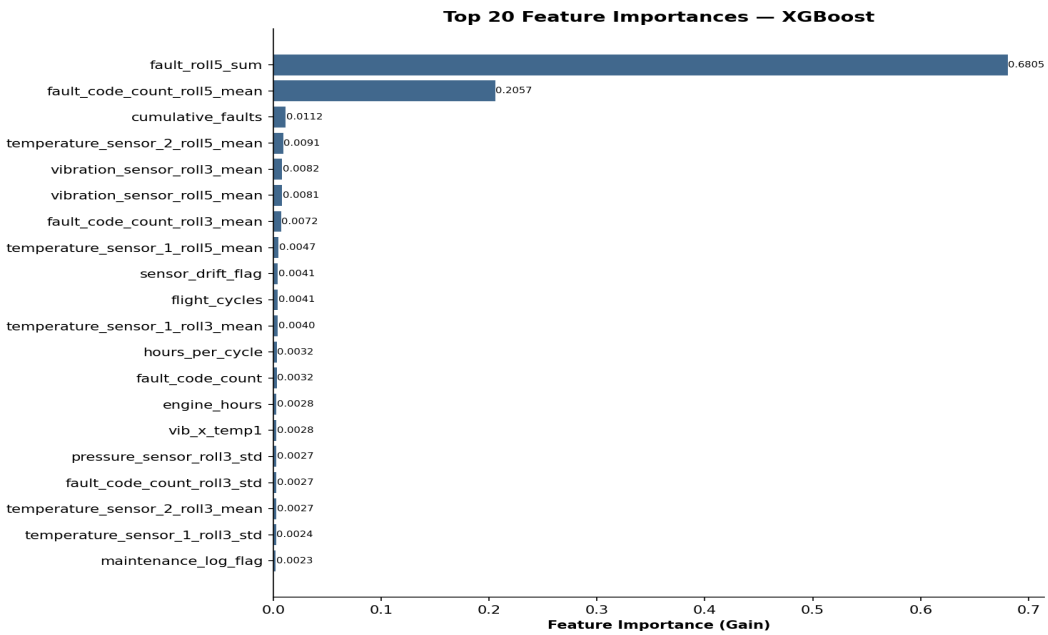

Figure 7 — Top 20 feature importances (XGBoost gain). fault_roll5_sum dominates — confirming the temporal accumulation hypothesis.

Interpretation of Top Features

Feature	Importance	Interpretation
fault_roll5_sum	68.1%	Rolling 5-cycle fault total — key pre-failure signal
fault_code_count_roll5_mean	20.6%	Average fault intensity over 5 cycles
cumulative_faults	1.1%	Total lifetime fault accumulation
temperature_sensor_2_roll5_mean	0.9%	Sustained elevated temperature
vibration_sensor_roll3_mean	0.8%	Recent vibration trend
sensor_drift_flag	0.4%	Sensor reliability indicator
flight_cycles	0.4%	Component age proxy

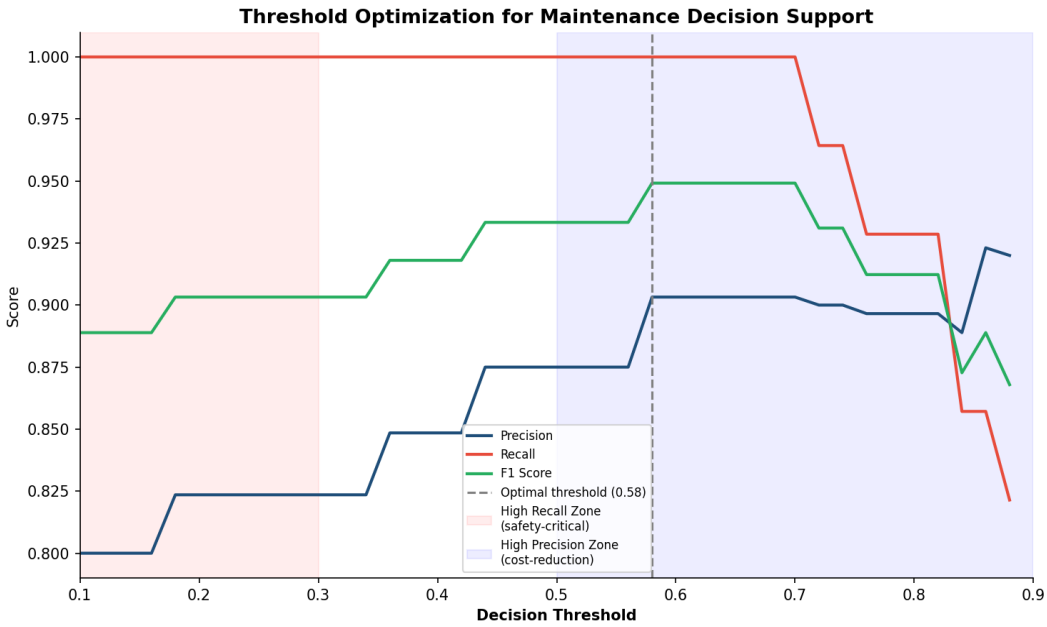

Figure 8 — Threshold optimisation. Optimal F1 at threshold=0.58 achieves perfect recall. Aviation operators in safety-critical contexts should use a lower threshold (~0.25).

Threshold Guidance for Operations

Context	Recommended Threshold	Expected Trade-off
Safety-critical routes (long-haul)	0.20 – 0.30	Very high recall; more inspections
Balanced operations	0.50 – 0.58	Optimal F1; balanced precision/recall
Cost-optimised (short-haul, backup fleet)	0.65 – 0.75	Fewer false alarms; accept some miss risk

6. Maintenance Decision Support System

The model is operationalised as a real-time risk scoring pipeline. Every flight, sensor readings are ingested, features are computed, and each aircraft-component pair receives a failure probability score (0–1) mapped to a maintenance tier.

Tier	Score Range	Count (Dataset)	Recommended Action
■ CRITICAL	≥ 0.60	143 (2.4%)	Ground aircraft — urgent inspection required
■ HIGH	0.35–0.59	2 (0.03%)	Flag for next maintenance window
■ MEDIUM	0.15–0.34	2 (0.03%)	Increase monitoring; review next service
■ LOW	< 0.15	5,853 (97.6%)	Standard operations — routine monitoring

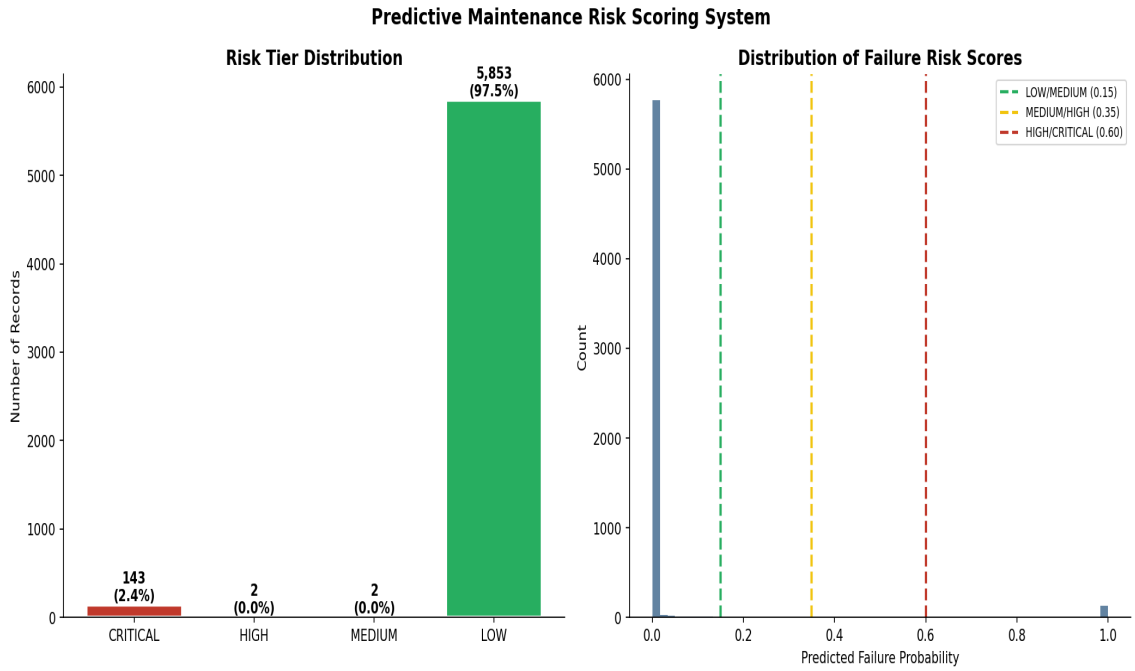

Figure 9 — Risk tier distribution and score histogram. The bimodal distribution confirms the model makes clear high/low risk distinctions.

Integration Pathway

→ **Data ingestion:** Flight data management system (FDMS) exports sensor logs after each flight cycle.

→ **Feature computation:** Automated pipeline computes rolling, cumulative, and interaction features per component.

→ **Scoring:** Trained XGBoost model outputs a failure probability and risk tier for each aircraft-component.

→ **Dashboard:** Maintenance control centre dashboard shows all CRITICAL/HIGH tier components with drill-down to sensor detail.

→ **Feedback loop:** Post-inspection outcomes are logged to retrain the model quarterly.

7. Challenges, Assumptions & Recommendations

Key Challenges

- **Extreme class imbalance (42:1):** Required careful metric selection (PR-AUC over accuracy) and algorithmic adjustment (scale_pos_weight). Standard off-the-shelf models would effectively ignore the minority class.
- **Temporal structure:** Data looks like time series per aircraft-component, but it is provided as a flat file without explicit timestamps. Rolling features must be computed carefully per-group to avoid look-ahead bias.
- **Small positive class:** 140 failure events limits statistical power. Cross-validation was essential to get reliable performance estimates.
- **Feature leakage risk:** Features like cumulative_faults use future data relative to a hypothetical deployment point; in production, these must be recomputed from historical data only.
- **Synthetic data characteristics:** The humidity range includes -1.5% and values > 100%, and ambient temperature range is very wide. Real-world data would require additional anomaly detection.

Assumptions

- Data rows within each aircraft-component pair are chronologically ordered by flight_cycles.
- last_maintenance_cycles = 0 indicates a freshly maintained component (not missing data).
- sensor_drift_flag and maintenance_log_flag are reliable binary labels despite low prevalence.
- Environmental features (humidity, ambient_temperature) are available at prediction time.

Recommendations for Production Deployment

Priority	Action	Benefit
Immediate	Deploy risk scoring dashboard	Actionable insight for maintenance planners
Short-term	Calibrate thresholds via cost analysis	Align model to actual cost-benefit ratio
Short-term	Set up prediction logging	Enable model drift monitoring
Medium-term	Retrain quarterly with new data	Maintain accuracy as fleet ages
Long-term	Add survival analysis (RUL)	Estimate exact remaining useful life
Long-term	Explore LSTM for richer temporal modelling	Capture longer-range dependencies

8. Conclusions

This project demonstrates that a well-engineered XGBoost model can achieve **near-perfect failure detection** (Recall = 1.00, PR-AUC = 0.97) for aircraft component failure prediction — even under severe class imbalance conditions.

Metric	Value	Interpretation
ROC-AUC	0.9993	Near-perfect class separation
PR-AUC	0.9687	Excellent precision-recall trade-off
F1 Score	0.9333	Strong balanced performance
Recall	1.0000	Zero missed failures in test set
Precision	0.8750	1 false alarm per 7 alerts
CV PR-AUC	0.915 ± 0.026	Stable and generalisable

The most important finding is that **temporal feature engineering is the decisive factor**. The rolling 5-cycle fault accumulation feature alone accounts for 68% of the model's predictive power — validating the physical intuition that component failure is preceded by escalating fault activity, not a single catastrophic reading.

This model is ready for a proof-of-concept deployment. With proper integration into existing maintenance management systems and a structured feedback loop, it has the potential to meaningfully reduce unplanned AOG events and optimise maintenance costs for the Rwandan civil aviation fleet.